



Smart Textile Manufacturing Quality Inspection and Production Optimization Platform

GIT AND DOCKER TECHNICAL ASSIGNMENT

Submitted in the partial fulfilment for the Course of

CSA1016

Software Engineering

to the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by

B Priyanka (192421206)

Under the supervision of

Dr. K. Anithadavamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai - 602105

August 2026

1. PROBLEM ANALYSIS

1.1 Problem Statement

The Smart Textile Manufacturing Quality Inspection and Production Optimization Platform is proposed to support textile manufacturing operations through digital monitoring and management. Textile manufacturing requires reliable quality inspection, production monitoring, and efficient management of manufacturing activities.

The proposed platform provides a centralized software environment for supporting textile quality inspection and production-related operations. The current implementation is a prototype developed using FastAPI with REST API functionality. Git is used for version control, while Docker and Docker Compose are used for containerization and deployment.

1.2 Project Objectives

The main objectives are:

1. Develop a software platform for smart textile manufacturing.
2. Provide a REST-based backend using FastAPI.
3. Maintain an organized and maintainable project repository.
4. Apply Git for version control.
5. Use feature-based branching for development.
6. Containerize the application using Docker.
7. Use Docker Compose for simplified deployment.
8. Provide API documentation through Swagger UI.
9. Validate application availability using a health-check endpoint.
10. Provide a foundation for future manufacturing monitoring and optimization features.

1.3 Functional Requirements

The current prototype provides the following functionality:

- REST API using FastAPI.
- /health endpoint for application health monitoring.
- Swagger API documentation through /docs.
- Docker-based application containerization.
- Docker Compose-based deployment.
- Git-based version control and branching.
- GitHub-based repository synchronization.

Future extensions may include AI-based fabric defect detection, production monitoring, machine monitoring, reporting, alerts, optimization, database integration, and authentication. These are future enhancements and are not claimed as implemented features of the current prototype.

1.4 Expected Outcomes

The expected outcome is a version-controlled and containerized backend prototype that can be consistently developed, deployed, and tested.

The implementation demonstrates Git repository management, feature-based development, branch integration, Docker image creation, Docker container execution, Docker Compose deployment, REST API availability, Swagger documentation, and successful health-check validation.

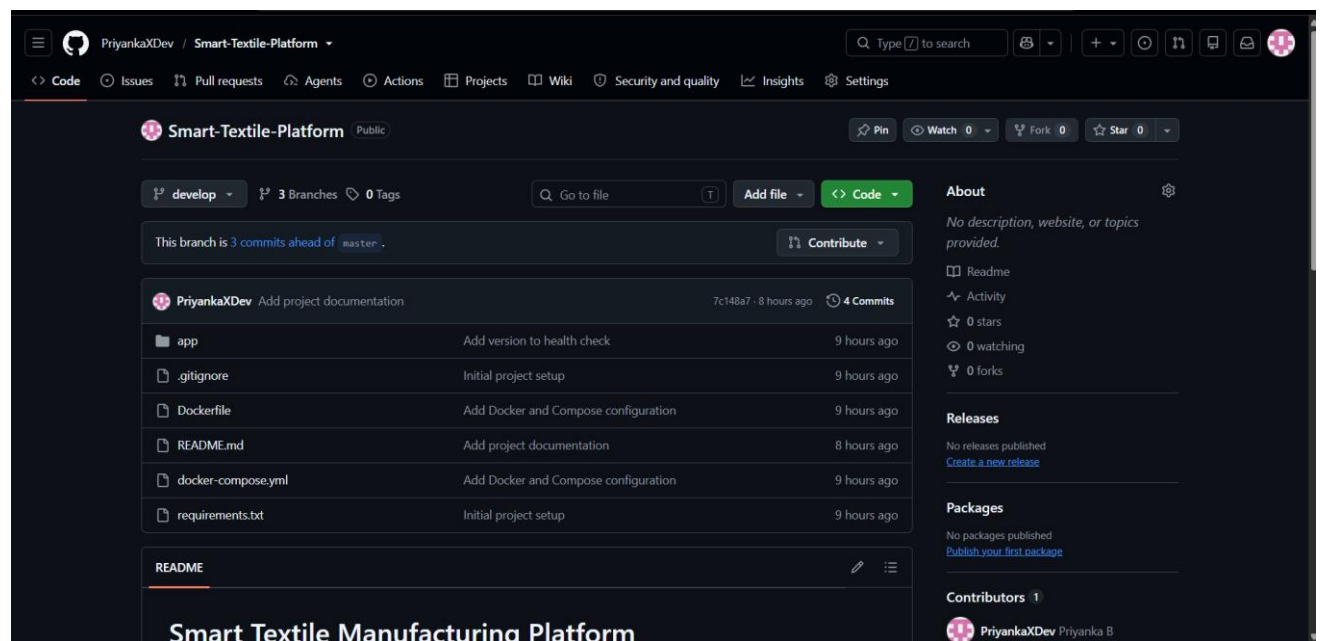


Figure 1.1 – Smart Textile Manufacturing Platform GitHub Repository

2. PROJECT STRUCTURE DESIGN

2.1 Repository Structure

The project repository separates application source code from configuration, dependency, documentation, and containerization files.

Smart-Textile-Platform/

```
|
|
|— app/
|   |— main.py
|
|
|— .gitignore
|— Dockerfile
|— docker-compose.yml
|— README.md
|— requirements.txt
```

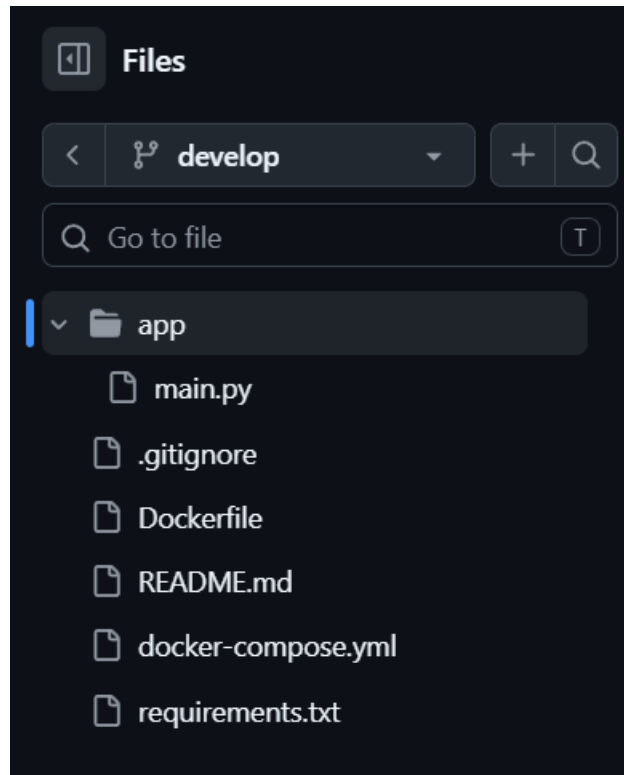


Figure 2.1 – Smart Textile Manufacturing Platform Project Structure

2.2 Purpose of Major Files

File/Folder	Purpose
app/	Contains application source code.
app/main.py	Defines the FastAPI application and API endpoints.
requirements.txt	Defines Python dependencies.
Dockerfile	Defines instructions for building the Docker image.
docker-compose.yml	Defines the Docker Compose deployment configuration.
.gitignore	Prevents unnecessary generated files from being tracked.
README.md	Provides project documentation and usage information.

The structure is suitable for the current prototype because it is simple and easy to understand. As the application grows, additional directories such as routers, services, models, schemas, and tests can be introduced.

3. GIT REPOSITORY INITIALIZATION

3.1 Repository Creation

Git was used as the version-control system for the project. The project was initialized as a Git repository and connected to the GitHub remote repository.

The initial project version was recorded using the commit:

Initial project setup

The GitHub repository was configured as the remote repository for synchronization.

```
PS D:\Projects\Smart-Textile-Platform> git log --oneline
dc19ad0 (HEAD -> master) Initial project setup
PS D:\Projects\Smart-Textile-Platform> git remote add origin https://github.com/PriyankaXDev/Smart-Textile-Platform.git
PS D:\Projects\Smart-Textile-Platform> git remote -v
origin https://github.com/PriyankaXDev/Smart-Textile-Platform.git (fetch)
origin https://github.com/PriyankaXDev/Smart-Textile-Platform.git (push)
PS D:\Projects\Smart-Textile-Platform> █
```

Figure 3.1 – Initial Git Commit and GitHub Remote Configuration

3.2 Git Initialization Workflow

The Git repository was initialized and connected to the GitHub remote repository. The initial project version was recorded using a commit, after which branches were created for organized development.

git init

git status

git add .

git commit -m "Initial project setup"

git remote add origin <repository-url>

git push -u origin master

git checkout -b develop

git checkout -b feature/health-check

The initial commit established the baseline version of the project. The develop branch was then used for integration, while the feature/health-check branch was used for isolated feature development.

3.3 Essential Git Files

The .git directory contains Git repository information including commit history, branches, and configuration. The .gitignore file prevents unnecessary generated files such as Python cache files from being tracked. The GitHub remote repository provides centralized storage and synchronization of project changes.

4. GIT BRANCHING STRATEGY

4.1 Selected Branching Strategy

A feature-based branching strategy was selected. The project contains:

master



develop



feature/health-check

The master branch represents the stable baseline, develop is used for ongoing integration, and feature/health-check is used for isolated feature development.

This strategy reduces the risk of directly modifying the stable branch and provides a simple workflow suitable for the current project.

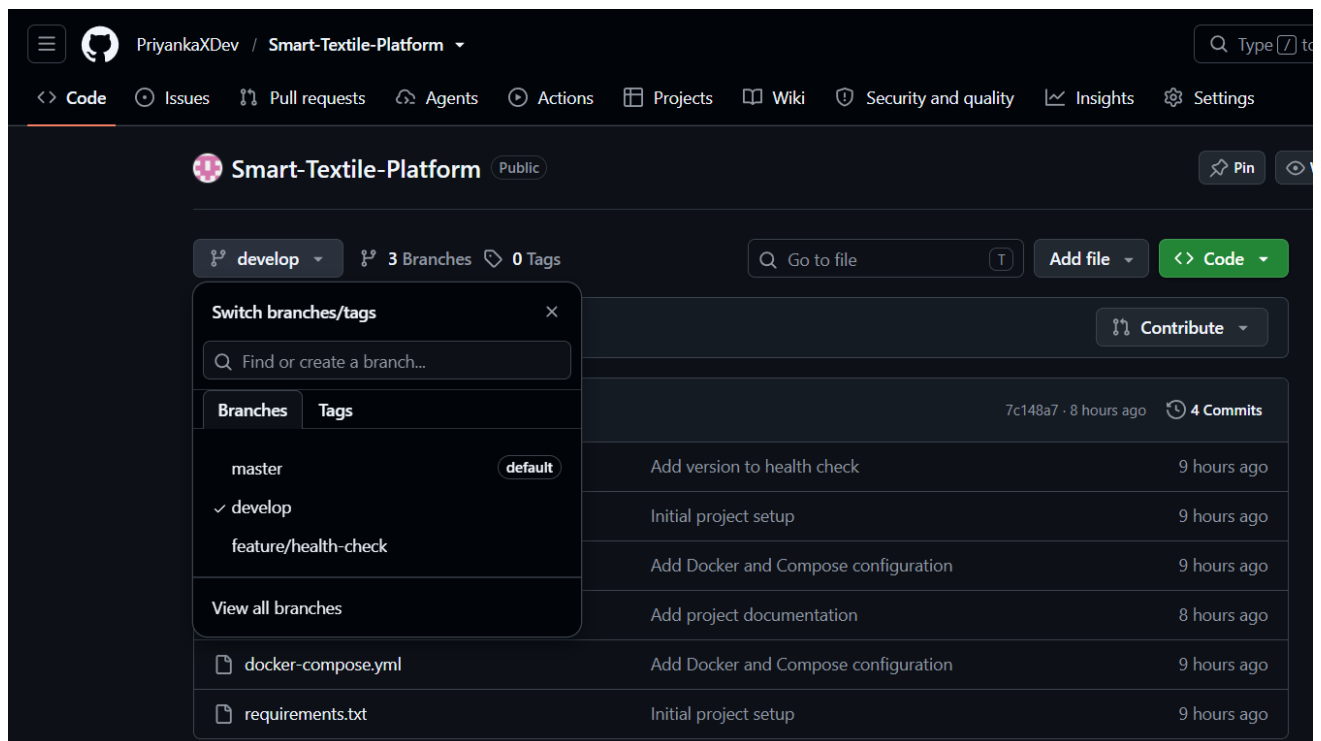


Figure 4.1 – Git Branching Structure

4.2 Branch Creation and Integration

The develop branch was created from master. The feature/health-check branch was then created for health-check development. After the feature and Docker configuration were completed, the feature branch was merged into develop.

5. VERSION CONTROL WORKFLOW

5.1 Git Workflow

The project followed the following workflow:

Feature Development

↓
git add
↓
git commit
↓
git push
↓
git merge
↓
develop

The workflow allows changes to be developed separately, recorded as commits, synchronized with GitHub, and integrated into the development branch.

```
PS D:\Projects\Smart-Textile-Platform> git merge feature/health-check
Updating dc19ad0..140580e
Fast-forward
 Dockerfile      | 13 ++++++++
 app/main.py     |  3 ++-
 docker-compose.yml |  8 +++++
 3 files changed, 23 insertions(+), 1 deletion(-)
PS D:\Projects\Smart-Textile-Platform> git log --oneline --decorate --graph --all
* 140580e (HEAD -> develop, origin/feature/health-check, feature/health-check) Add Docker and Compose configuration
* 1ed67b9 Add version to health check
* dc19ad0 (origin/master, master) Initial project setup
PS D:\Projects\Smart-Textile-Platform>
```

Figure 5.1 – Feature Branch Merged into Develop

5.2 Major Git Operations

Git Operation	Purpose
git add	Stages project changes.
git commit	Records a version of the changes.
git push	Synchronizes local changes with GitHub.
git checkout	Switches or creates branches.
git merge	Integrates feature changes into another branch.

The feature/health-check branch was successfully merged into develop using a fast-forward merge.

No merge conflict occurred during the demonstrated integration.

6. COMMIT HISTORY ANALYSIS

6.1 Commit History

The project maintains a chronological commit history representing the major stages of development. The recorded commits include the initial project setup, health-check implementation, Docker and Compose configuration, and project documentation.

Initial project setup

↓

Add version to health check

↓

Add Docker and Compose configuration

↓

Add project documentation

```
PS D:\Projects\Smart-Textile-Platform> git add app/main.py
PS D:\Projects\Smart-Textile-Platform> git commit -m "Add version to health check"
[feature/health-check 1ed67b9] Add version to health check
 1 file changed, 2 insertions(+), 1 deletion(-)
PS D:\Projects\Smart-Textile-Platform> git log --oneline --decorate
1ed67b9 (HEAD -> feature/health-check) Add version to health check
dc19ad0 (origin/master, master) Initial project setup
PS D:\Projects\Smart-Textile-Platform> █
```

Figure 6.1 – Project Commit History

6.2 Commit Message Quality

The commit messages describe the purpose of the corresponding changes. Examples include Add version to health check, Add Docker and Compose configuration, and Add project documentation. These messages make the development history easier to understand and maintain.

The commit history provides traceability and allows previous project states to be reviewed when required.

7. DOCKER ENVIRONMENT DESIGN

7.1 Need for Docker

Docker is suitable for the project because the FastAPI application depends on a specific Python runtime and application dependencies. Without containerization, differences in Python versions or dependency configurations could cause inconsistent behavior between development environments. Docker provides an isolated and reproducible environment for executing the application.


```
PS D:\Projects\Smart-Textile-Platform> docker version
Client:
Version:      29.6.2
API version:  1.55
Go version:   go1.26.5
Git commit:   dfc4efb
Built:        Thu Jul 16 16:14:59 2026
OS/Arch:      windows/amd64
Context:      desktop-linux

Server: Docker Desktop 4.85.0 (235549)
Engine:
Version:      29.6.2
API version:  1.55 (minimum version 1.40)
Go version:   go1.26.5
Git commit:   3d80467
Built:        Thu Jul 16 16:12:20 2026
OS/Arch:      linux/amd64
Experimental: false
containerd:
Version:      v2.2.5
GitCommit:    e53c7c1516c3b2bff98eb76f1f4117477e6f4e66
runc:
Version:      1.3.6
GitCommit:    v1.3.6-0-g491b69ba
docker-init:
Version:      0.19.0
GitCommit:    de40ad0
PS D:\Projects\Smart-Textile-Platform>
```

Figure 7.1 – Docker Environment Verification

7.2 Components Requiring Containerization

The current prototype contains one primary application component:

FastAPI Application



Python Runtime



Docker Container

The application source code and Python dependencies are packaged into the Docker image. Docker Compose is then used to build and start the application container.

8. DOCKERFILE DEVELOPMENT

8.1 Dockerfile

The project uses the following Dockerfile:

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app ./app
```

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

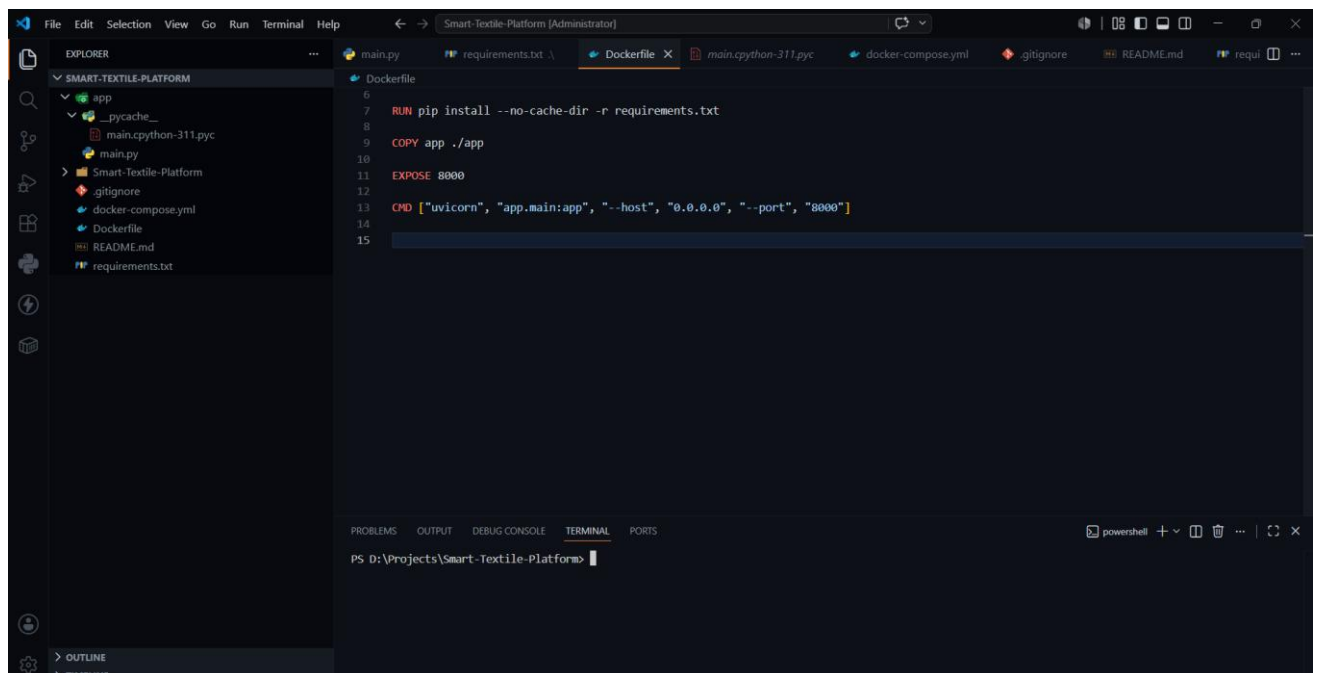


Figure 8.1 – Dockerfile Configuration

8.2 Dockerfile Instructions

Instruction	Purpose
FROM	Provides the Python 3.11 Slim base image.
WORKDIR	Sets /app as the container working directory.
COPY requirements.txt	Copies the dependency file into the container.
RUN	Installs the required Python packages.
COPY app	Copies the application source code.
CMD	Starts the FastAPI application using Uvicorn on port 8000.

The Dockerfile therefore provides a reproducible procedure for building the application environment.

9. DOCKER COMPOSE DESIGN

9.1 Docker Compose Configuration

Docker Compose simplifies the building and execution of the application container. The current configuration defines the Smart Textile application service and exposes port 8000.

services:

smart-textile:

build:

context: .

dockerfile: Dockerfile

container_name: smart-textile-compose

ports:

- "8000:8000"

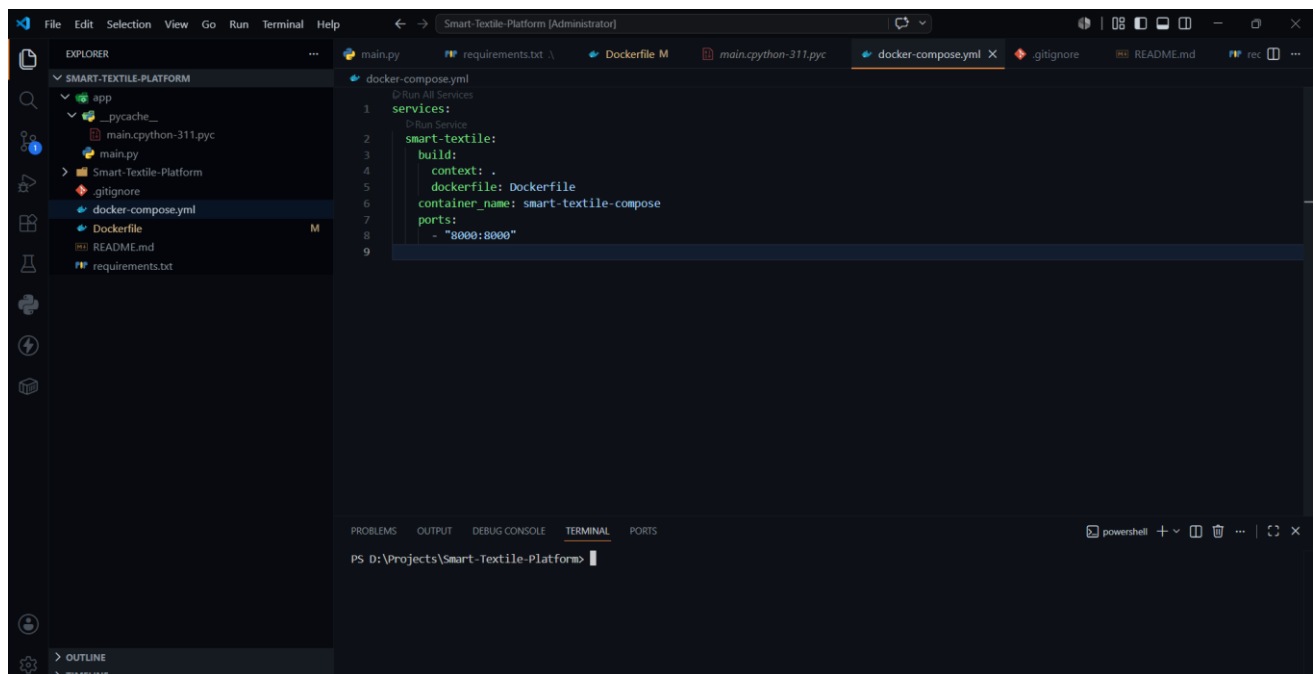


Figure 9.1 – Docker Compose Configuration

9.2 Service and Network Configuration

The main service is smart-textile. It builds the application using the Dockerfile and maps port 8000 of the host to port 8000 of the container.

The application can therefore be accessed through:

<http://localhost:8000>

Docker Compose automatically creates a project network. Since the current prototype contains only one service, complex inter-service communication is not required.

No external volumes, environment variables, or service dependencies are currently required by the prototype.

10. CONTAINER DEPLOYMENT AND TESTING

10.1 Application Deployment

The application was built and started using:

```
docker compose up -d --build
```

This command builds the Docker image, creates the Docker network, creates the container, and starts the application.

10.2 Container Verification

The running container was verified using:

```
docker compose ps
```

The container was displayed with an Up status and the following port mapping:

```
0.0.0.0:8000->8000/tcp
```

```
PS D:\Projects\Smart-Textile-Platform> docker images smart-textile-platform
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
smart-textile-platform:1.0          05b2ea7d774e                     226MB       54.8MB
PS D:\Projects\Smart-Textile-Platform> docker run -d --name smart-textile-container -p 8000:8000 smart-textile-platform:1.0
0fe9e526392c3b586606d439c25613965c21c5cb1e484fbc84bb253f6456e40e
PS D:\Projects\Smart-Textile-Platform> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
0fe9e526392c   smart-textile-platform:1.0          "uvicorn app.main:ap..." 6 seconds ago  Up 5 seconds  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp  smart-textile-container
PS D:\Projects\Smart-Textile-Platform>
```

Figure 10.1 – Docker Compose Container Status

10.3 API Testing

The FastAPI Swagger interface was accessed through:

```
http://localhost:8000/docs
```

The /health endpoint was executed using Swagger UI.

The response was:

```
{
  "status": "healthy",
  "service": "Smart Textile Manufacturing Platform",
  "version": "1.0.0"
}
```

The HTTP response code was:

```
200 OK
```

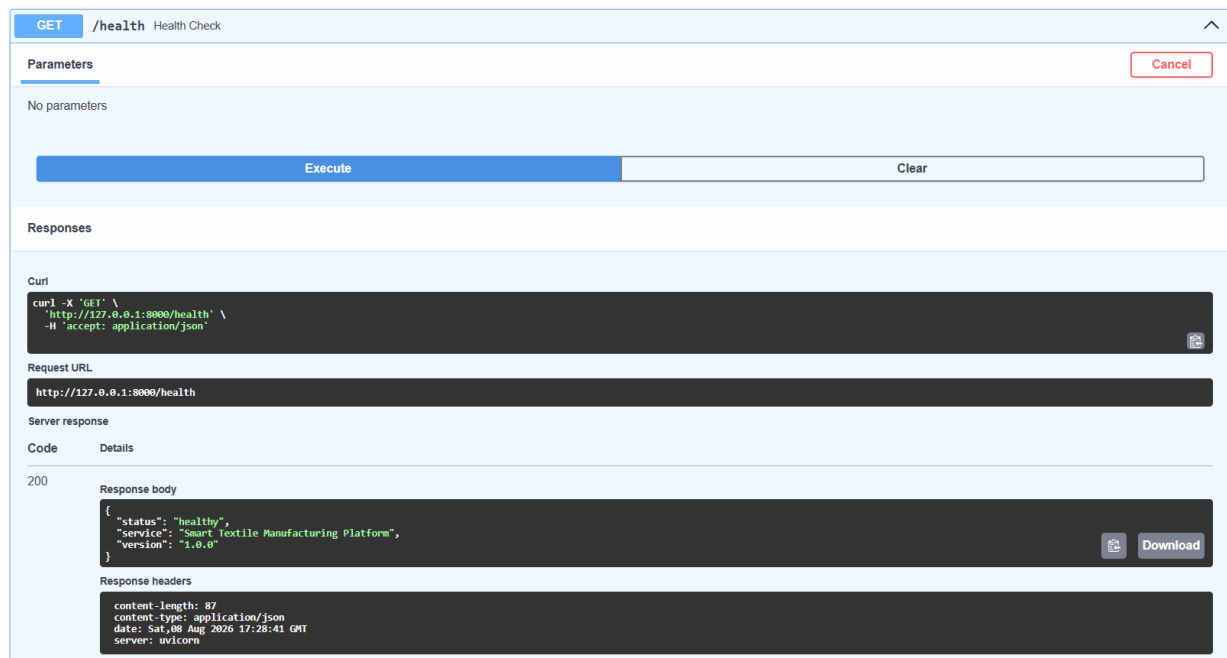


Figure 10.2 – Successful Health Check API Test

10.4 Validation

Docker Compose logs confirmed successful application startup and API request processing. The logs showed successful requests for /docs, /openapi.json, and /health.

The deployment and testing results demonstrate that the application can be built, started inside a Docker container, accessed through the exposed port, and validated through its health-check API.

```
PS D:\Projects\Smart-Textile-Platform> docker compose logs
smart-textile-compose | INFO:      Started server process [1]
smart-textile-compose | INFO:      Waiting for application startup.
smart-textile-compose | INFO:      Application startup complete.
smart-textile-compose | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
smart-textile-compose | INFO:      172.19.0.1:49344 - "GET /docs HTTP/1.1" 200 OK
smart-textile-compose | INFO:      172.19.0.1:49344 - "GET /openapi.json HTTP/1.1" 200 OK
smart-textile-compose | INFO:      172.19.0.1:53974 - "GET /health HTTP/1.1" 200 OK
PS D:\Projects\Smart-Textile-Platform>
```

Figure 10.3 – Docker Container Logs and API Request Validation

11. BENEFITS OF GIT AND DOCKER

11.1 Benefits of Git

Git provides:

- **Version Management:** Records project changes through commits.
- **Branch-Based Development:** Allows features to be developed independently.
- **Collaboration:** GitHub provides a centralized remote repository.
- **Change Tracking:** Commit history records project evolution.
- **Recovery:** Previous versions can be reviewed when required.

11.2 Benefits of Docker

Docker provides:

- **Portability:** The application runs in a consistent environment.
- **Dependency Isolation:** Dependencies are separated from the host environment.
- **Reproducibility:** The Dockerfile can recreate the application environment.
- **Deployment Simplicity:** Docker Compose simplifies application deployment.
- **Maintainability:** Container configuration can be maintained as project code.

Together, Git and Docker improve version management, collaboration, portability, reproducibility, and deployment consistency.

12. CHALLENGES AND IMPROVEMENTS

12.1 Challenges Encountered

The main challenges encountered were:

1. **Git Remote Configuration:** The GitHub remote configuration initially required correction before successful synchronization.
2. **Branch Management:** Separate development and feature branches had to be created and integrated.
3. **Docker Configuration:** The Dockerfile and Compose configuration had to be correctly configured for application execution.
4. **Port Configuration:** Port 8000 had to be correctly mapped between the host and container.
5. **Container Validation:** The deployed application had to be verified through Docker status, logs, and API testing.

12.2 Future Improvements

Future improvements include:

1. Database integration.
2. Authentication and authorization.
3. Further modularization of the application.
4. Automated unit and integration testing.
5. CI/CD automation.
6. AI-based fabric defect detection.
7. Production and machine monitoring.
8. Reporting and dashboard functionality.
9. Additional Docker Compose services such as a database.
10. Cloud deployment.

CONCLUSION

The Git and Docker implementation demonstrates the practical application of version control and containerization concepts to the Smart Textile Manufacturing Platform.

Git was used for repository management, branching, commits, feature development, merging, and synchronization with GitHub. Docker was used to package the FastAPI application and its dependencies into a reproducible environment, while Docker Compose simplified deployment.

The successful container execution and HTTP 200 response from the /health endpoint demonstrate that the current prototype can be built, deployed, and validated successfully. The project provides a foundation for future database integration, automated testing, CI/CD, AI-based defect detection, production monitoring, and additional manufacturing functionality.

GITHUB REPOSITORY

Repository:

<https://github.com/PriyankaXDev/Smart-Textile-Platform>